

Naive Bayes classifier for Fake News recognition

Advanced Statistics for Physics Analysis
Prof. Garfagnini



Outline

- What is the problem?
- Datasets and preprocessing
- The algorithm and Its Implementation in R
- Evaluating the results
- Conclusions



What are fake news? And why does it matter?

- **Fake news:** false stories produced with the intent of confusing or deceiving people.
- They represent a pressing issue, as the widespread use of **social media** has increased the speed at which false information spreads.
- We will tackle this by developing an algorithm for the prediction of fake news, based on text classification with the **Naive Bayes** method.



Datasets

- Fake News Content Detection
 - Train
 - Test
 - Sample submission
- Fake News Classification
 - Train
 - Test
 - Evaluation

The Kaggle logo, consisting of the word "kaggle" in a lowercase, blue, sans-serif font.

Data Usage Strategy

- For the first the set we use only the training file because it's the one that is labeled. To evaluate our model, we split it into training and validation sets.
- For the second dataset, we use the evaluation data. This decision can be justified by our intention to test the method on a binary classification dataset with a similar number of samples as the first dataset.
- We split our dataset into 80% for training and 20% for evaluating the performance of the model.

What can we say about our data?

Fake News Content Detection

- 10240 labeled sample in Train
- 3 columns
 - Labels
 - Text
 - Text_Tag
- 6 different category labels
 - True - 5
 - Not-Known - 4
 - Mostly-True - 3
 - Half-True - 2
 - False - 1
 - Barely-True - 0

The dataset is clean, with no missing values or need for data type conversion.

What can we say about our data?

Fake News Classification - Raw data

- 36454 labeled sample in evaluation set
- Binary classification
 - Fake(0)
 - True(1)
- 1 column, but contains:
 - Num(index)
 - Title
 - Text
 - Label

What can we say about our data?

Fake News Classification - Raw data Issues and Solutions

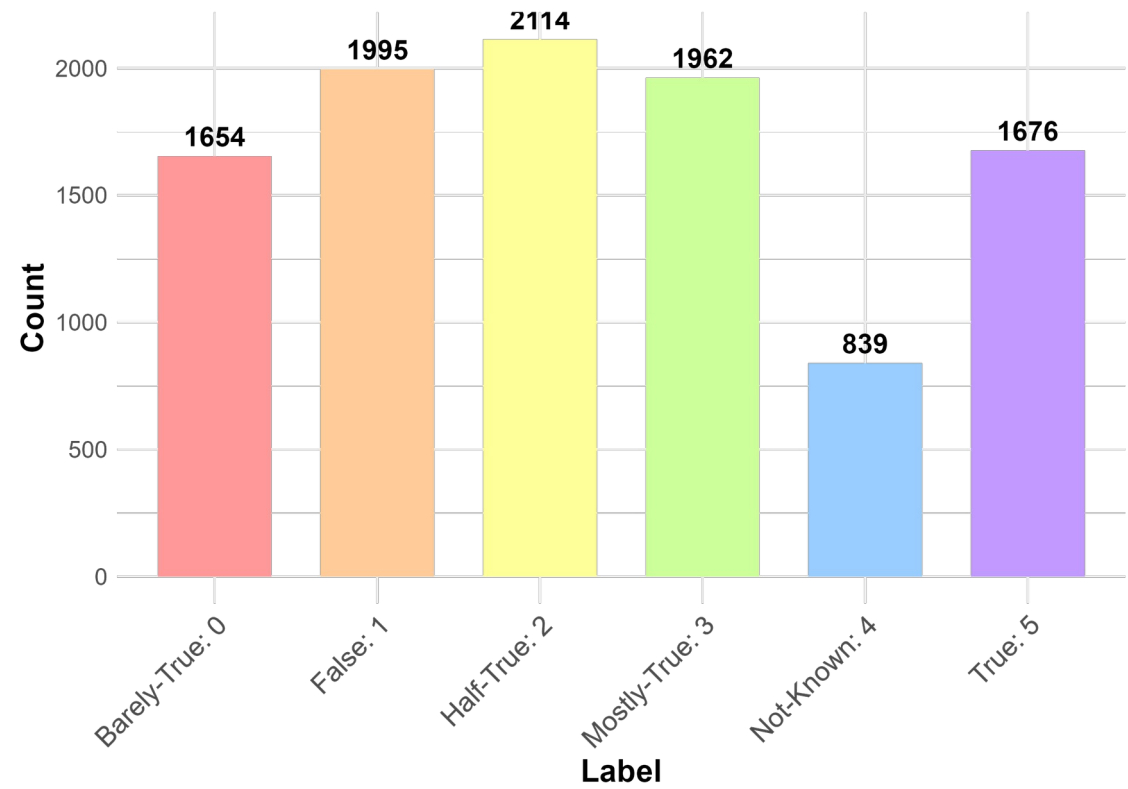
- The dataset is not well-formatted.
We split it's column into four separate columns for better usability.
- Some rows contain missing or invalid labels, which we removed during cleaning.
- The Num column serves only as an index and is not useful for analysis, so we dropped it.
- We converted the Labels column to numeric to ensure consistency with the first dataset and to make it more suitable for modeling.
- We reordered the columns to Labels, Text, and Text_Tag to improve clarity and maintain consistency.
- We removed Rows with Text entries shorter than 20 characters , as such short texts are unlikely to provide enough information for effective classification.

What can we say about our data?

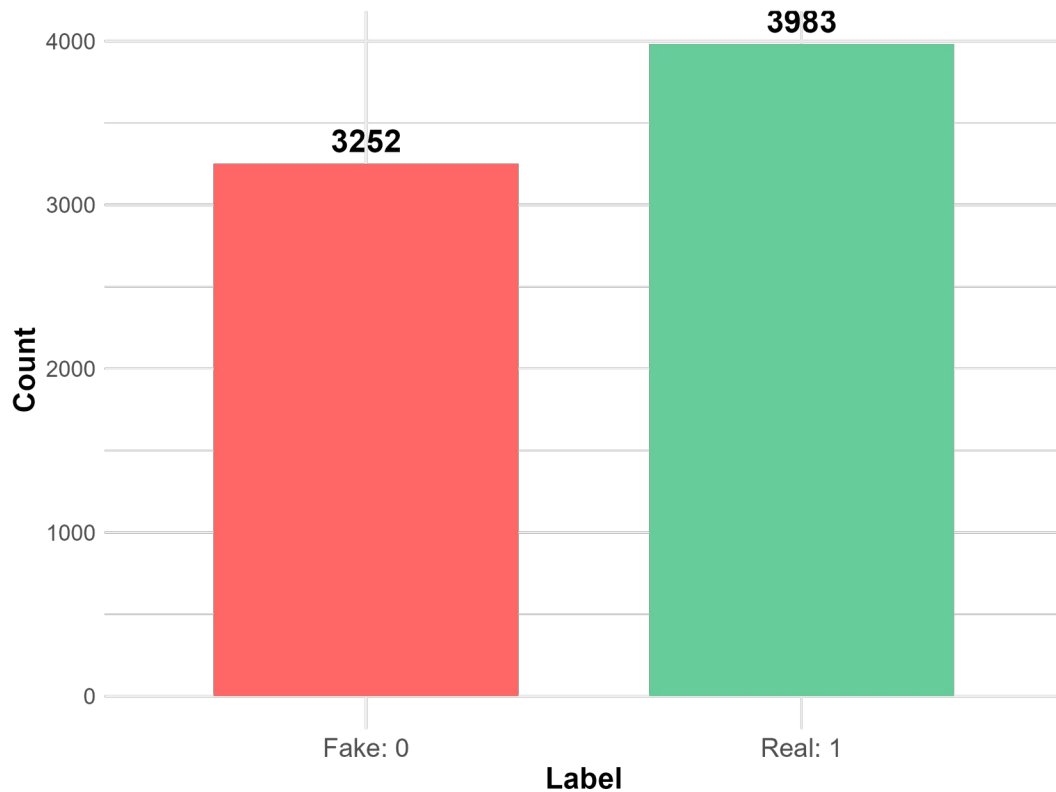
Fake News Classification - Cleaned

- 21662 labeled sample
- 3 columns
 - Labels
 - Text
 - Text_Tag
- Binary classification
 - Fake(0)
 - True(1)

Distribution of Labels (Multi-class Data)

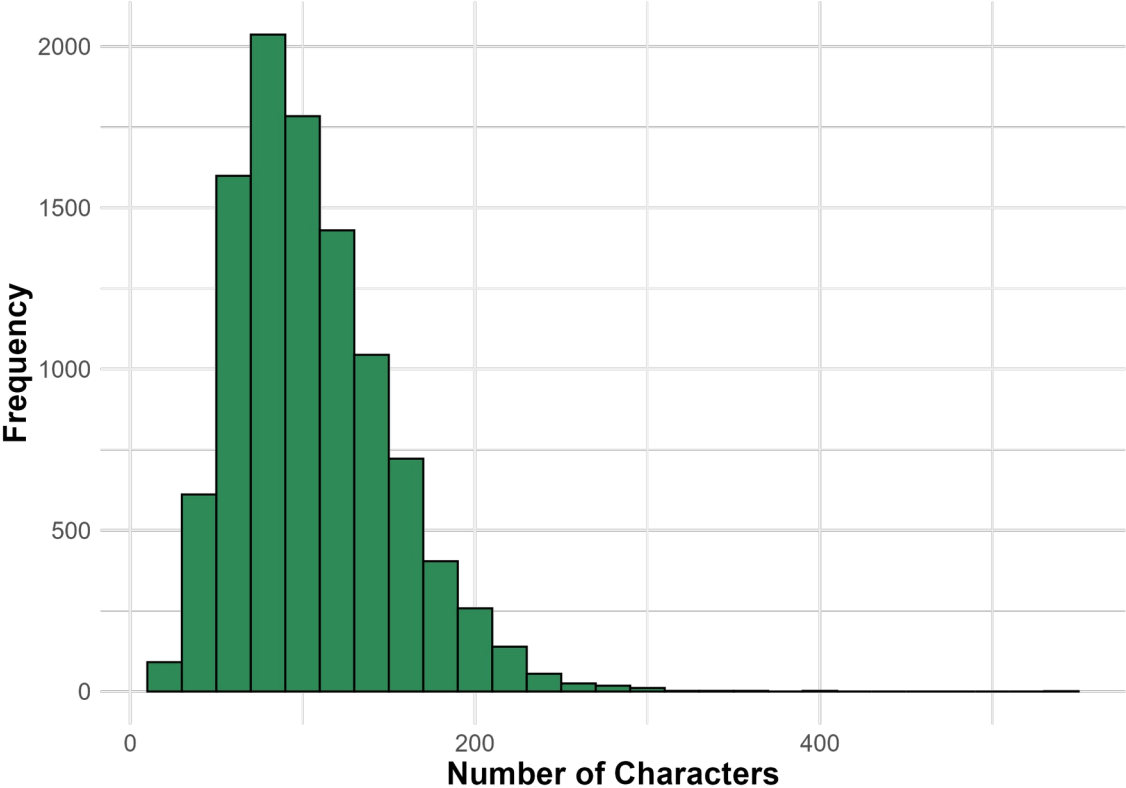


Distribution of Labels (Binary-class Data)

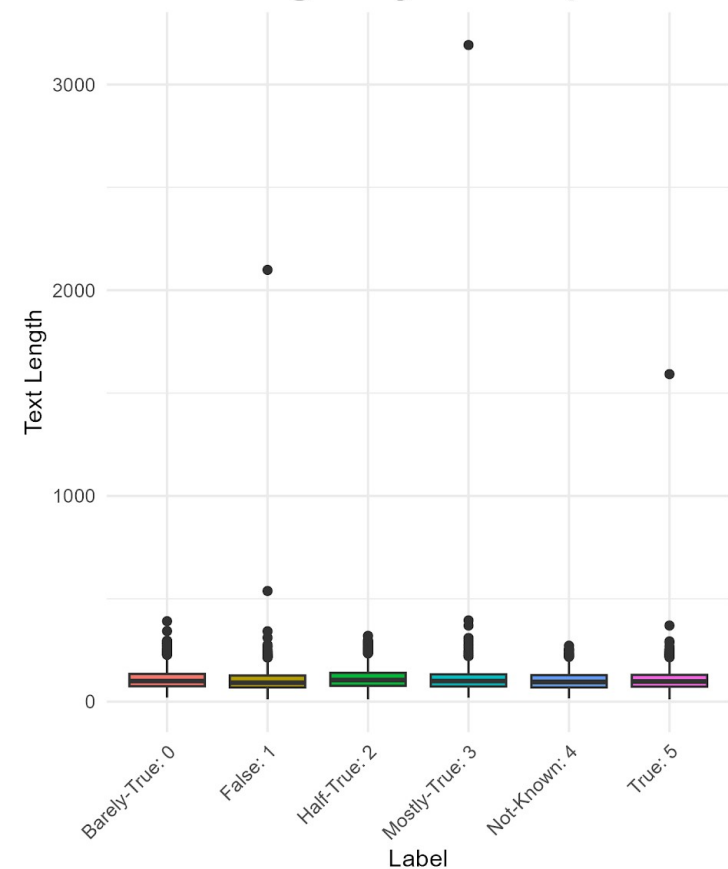


Text Length Analysis (Multi-class Data)

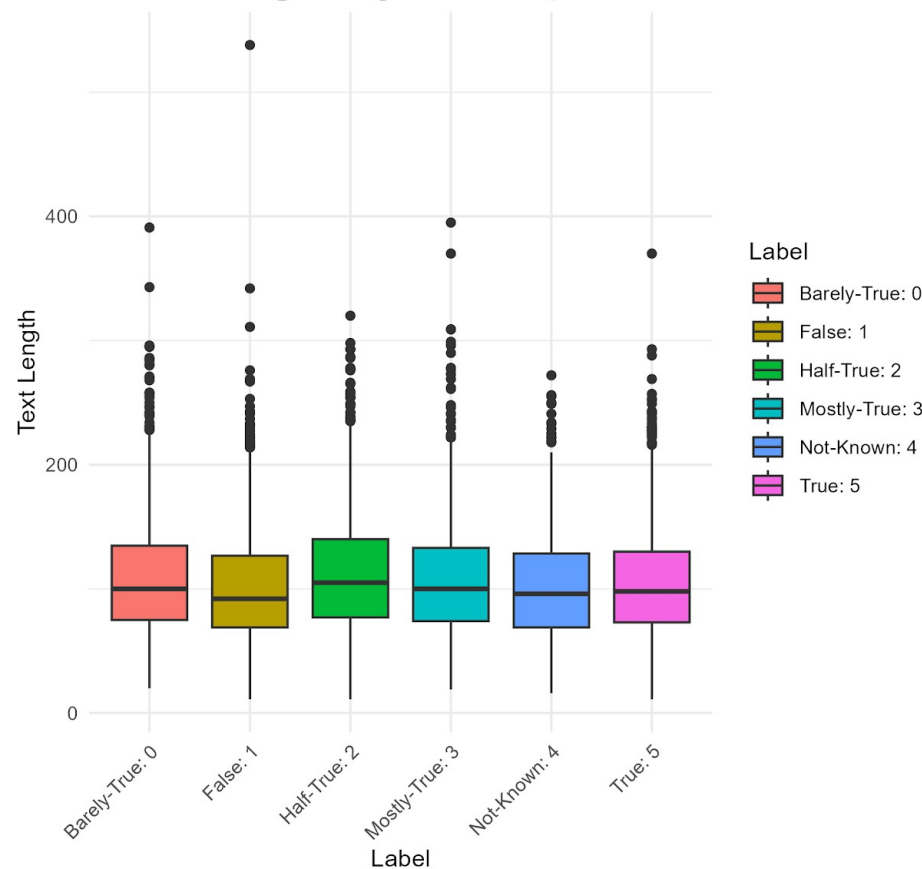
Text Length Distribution
in multi-class Data
(≤ 1000 Characters)



Text Length by Label (All Data)

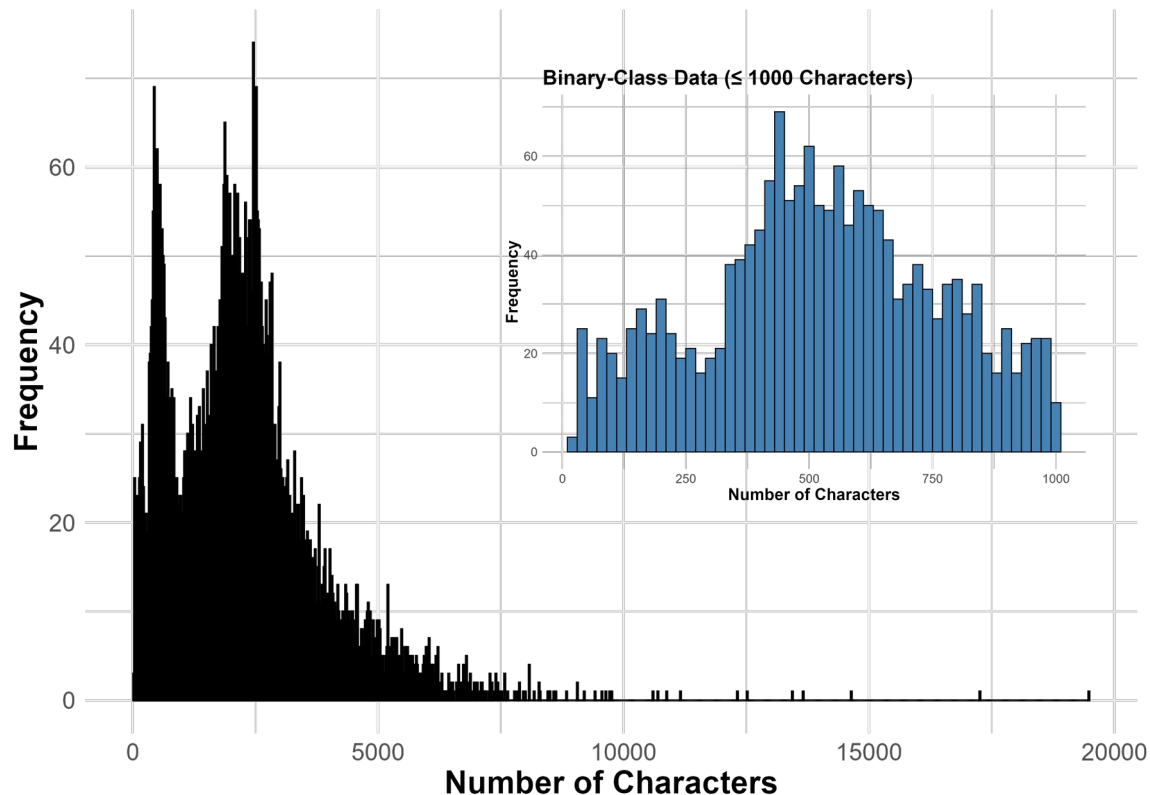


Text Length by Label (< 1000 Characters)

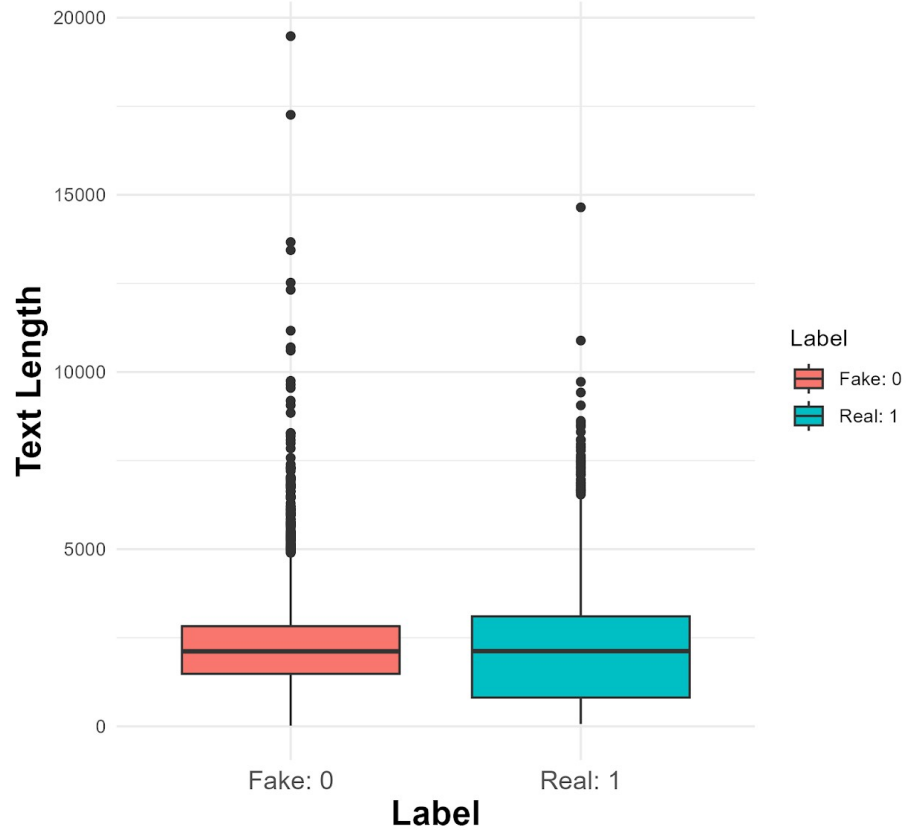


Text Length Analysis (Binary-class Data)

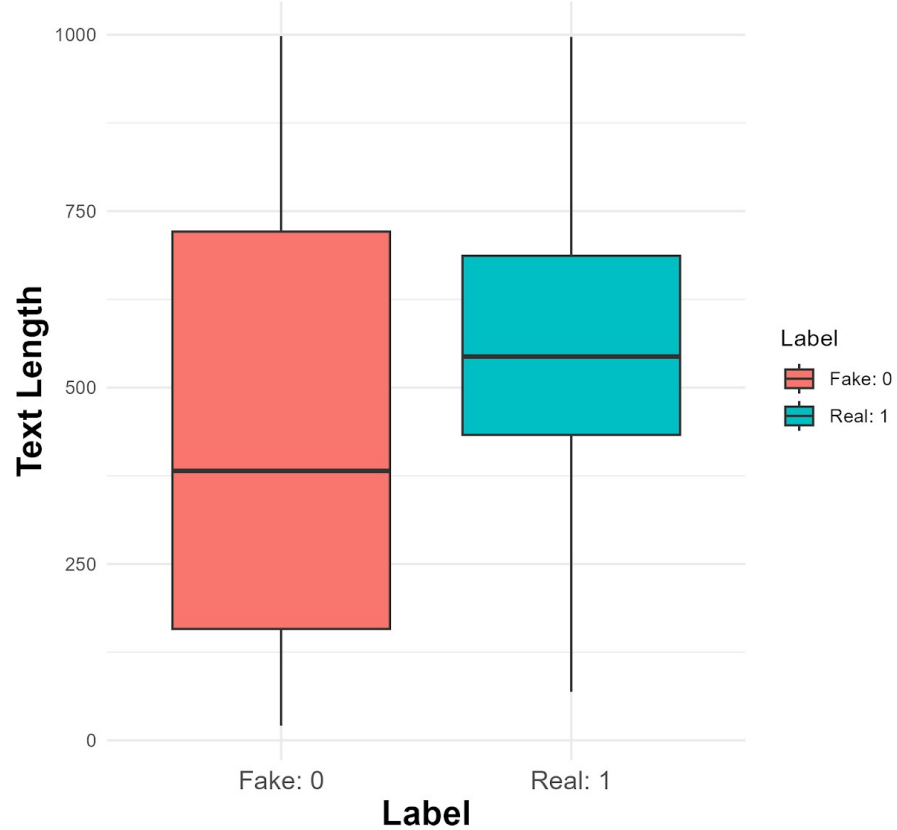
Compared to the multiclass dataset, the binary class texts are generally longer. Some samples are extremely large, and we might consider treating them as outliers.



Text Length by Label (All Data)



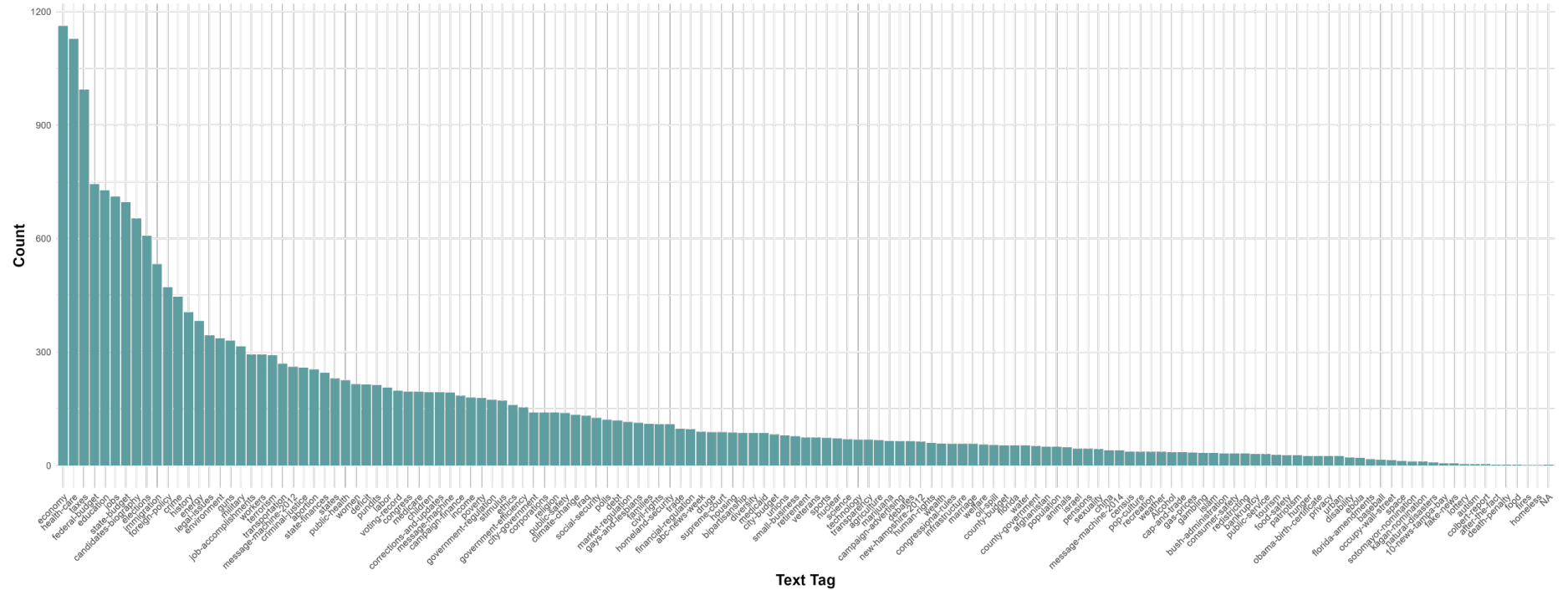
Text Length by Label (< 1000 Characters)



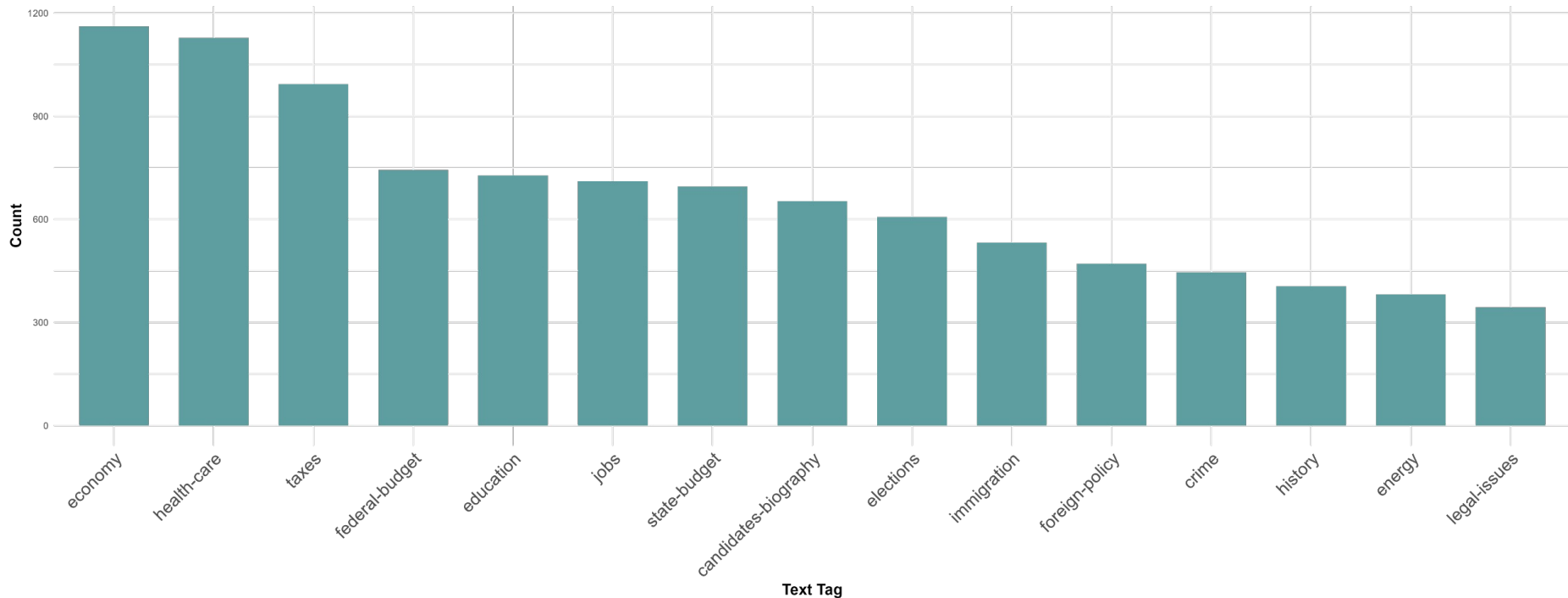
What about the “Third” column

- In the Naive Bayes modeling approach, we focused solely on the main Text and Labels, excluding other features like the news title or text_tag.
- While this approach simplifies the model, it may overlook valuable contextual cues.
- More complex models, such as Neural Networks, could potentially leverage these additional features to improve classification accuracy by capturing deeper semantic relationships.

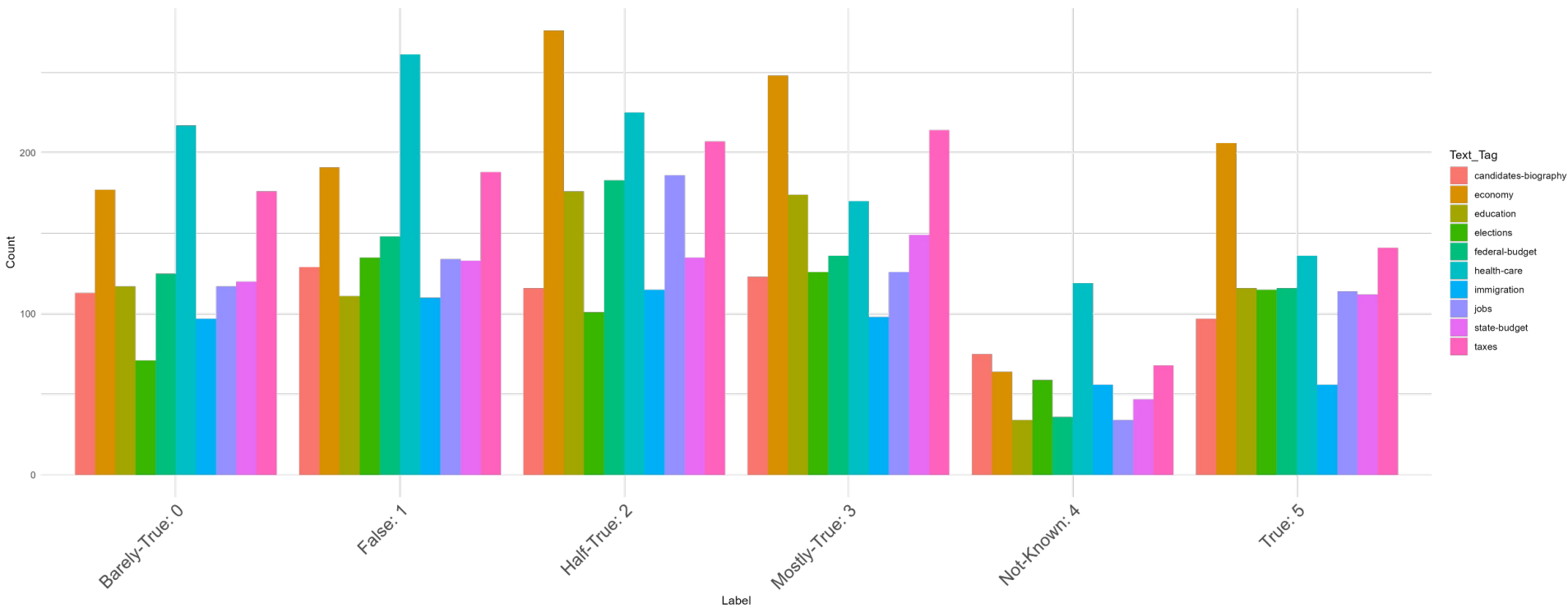
Distribution of News Tags in Multi-class data



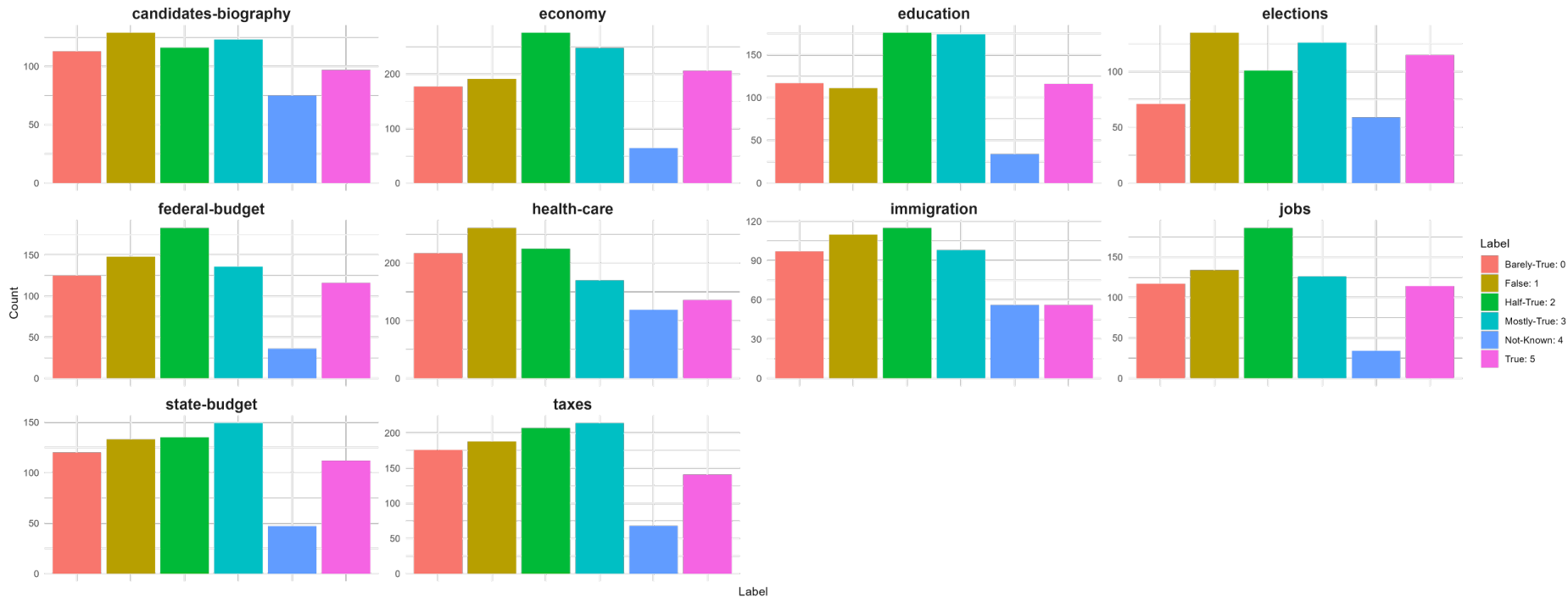
Top 15 News Tags in Multi-class data



Label Distribution per Category (Text_Tag) - Top 10



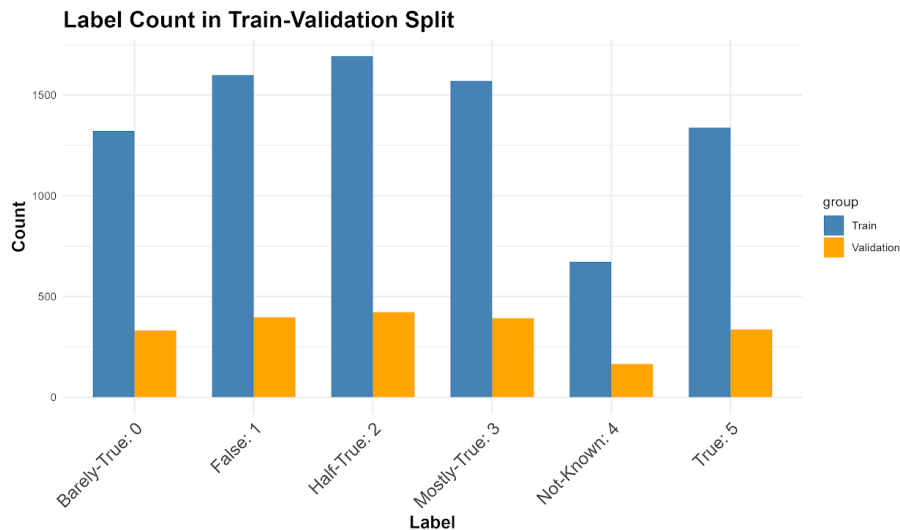
Label Distribution per Top 10 Category - Faceted Plots



Train-validation Split & Preprocessing

- Stratified Sampling (caret):

Ensures balanced class distribution in train-validation split.



- Text Preprocessing & Encoding:

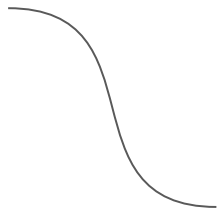
Clean and standardize text, then convert to numeric format for compatibility with Naive Bayes.



How can we prepare text sequences for classification?

we defined 4 different preprocessing steps and NLP techniques on our data and tried to compare their results and performances.

- Basic processing
- Stopword removal
- Stemming
- Lemmatization



- Clean & Tokenizing:
 - Removing punctuation
 - Replacing multiple spaces with a single space
 - Converting all text to lowercase
 - Splitting the text into individual word tokens
- Numerical Representation
 - Identifies unique terms
 - Assigns each term a unique numeric ID starting from 1

```
[[1]]
[1] "says"      "the"      "annies"   "list"     "political" "group"
[7] "supports"  "third"    "trimester" "abortions" "on"        "demand"

[[2]]
[1] "hillary" "clinton" "agrees"  "with"  "john"  "mccain" "by"
[8] "voting"  "to"      "give"    "george" "bush"  "the"    "benefit"
[15] "of"      "the"     "doubt"   "on"     "iran"
```

Developing More Advanced Text Processing

- **Stopword Removal:**

Performs basic preprocessing and removes short, common stopwords (under 4 characters), targeting high-frequency function words such as “the”, “and”, and “not”.

```
[[1]]
[1] "says"      "annies"    "list"      "political" "group"     "supports"
[7] "third"     "trimester" "abortions" "demand"
```



```
[[2]]
[1] "hillary" "clinton" "agrees" "with" "john" "mccain" "voting"
[8] "give"    "george"  "bush"   "benefit" "doubt" "iran"
```

Developing More Advanced Text Processing

- **Stemming**

In addition to basic preprocessing, we applied stemming to reduce words to their root form.

This helps unify word variants, reducing feature space sparsity. While effective for general meaning, it may produce non-standard word forms.

"running" → "run"

note: not always a real word

"better" → "better"

"flies" → "fli"

"played" → "play"

"studies" → "studi"

"connection" → "connect"

"says" → "sai"

Stemming

- **Purpose:**

- Groups different forms of a word (e.g., "run", "running", "runs") into a single token.
- Reduces vocabulary size.
- Helps machine learning models treat related words as the same feature.

- **Limitation:**

Stemming can be aggressive and may produce non-dictionary forms (e.g., "studies" → "studi").

- **Common Algorithms:**

- Snowball Stemmer
- Porter Stemmer
- Lancaster Stemmer

```
[[1]]
[1] "sai"      "anni"     "list"     "polit"    "group"    "support"  "third"
[8] "trimest"  "abort"    "demand"

[[2]]
[1] "hillari"  "clinton"  "agre"     "with"     "john"     "mccain"   "vote"
[8] "give"     "georg"    "bush"     "benefit"  "doubt"    "iran"
```

Developing More Advanced Text Processing

- **Lemmatization**

is the process of reducing words to their base or dictionary form, known as the lemma.

Unlike stemming, lemmatization considers the context and grammar of the word, ensuring the resulting form is a valid word.

```
[[1]]
[1] "say"          "annies"      "list"        "political"   "group"       "support"
[7] "3"           "trimester"  "abortion"    "demand"
```



```
[[2]]
[1] "hillary" "clinton" "agree"    "with"    "john"    "mccain" "vote"
[8] "give"    "george"  "bush"     "benefit" "doubt"   "iran"
```

Lemmatization

- Pros:
 - Linguistically accurate: Returns real words
 - Context-aware: Considers part of speech and grammar.
 - Reduces noise: Improves consistency in features for text models.
- Cons:
 - More complex and Slower
 - Language-dependent: Requires a lemmatizer for each language.

"running" → "run"

"was" → "be"

"children" → "child"

"studies" → "study"

"flies" → "fly"

"driven" → "drive"

"better" → "good"

"says" → "say"

"happiest" → "happy"

Building Vocabulary & feature selection

4 different preprocessing pipeline:

- Basic (punctuation, multi-space, lowercase, tokenizing)
- Basic + stopwords removal (“the”, “and”, “not”)
- Basic + stopwords removal + Stemming
- Basic + stopwords removal + Lemmatization

Number of unique vocabularies(tokens):

- Vocabulary Size (Basic Processing): 11164
- Vocabulary Size (Basic + Stopword removal): 11044
- Vocabulary Size (Basic + Stopword removal + stemming): 7617
- Vocabulary Size (Basic + Stopword removal + lemmatization): 8317

The Naive Bayes method for text classification

- One way to identify fake news is to rephrase the problem in terms of **text classification**:
 - We are given some text $x \in \mathbb{X}$, where $\mathbb{X}$ is our *input space*, and a fixed set of classes $\mathbb{C} = \{c_1, c_2, \dots, c_j\}$.

The Naive Bayes method for text classification

- One way to identify fake news is to rephrase the problem in terms of **text classification**:
 - We are given some text $x \in \mathbb{X}$, where $\mathbb{X}$ is our *input space*, and a fixed set of classes $\mathbb{C} = \{c_1, c_2, \dots, c_j\}$.
 - For instance, the classes may represent different levels of truthfulness.

The Naive Bayes method for text classification

- One way to identify fake news is to rephrase the problem in terms of **text classification**:
 - We are given some text $x \in \mathbb{X}$, where $\mathbb{X}$ is our *input space*, and a fixed set of classes $\mathbb{C} = \{c_1, c_2, \dots, c_j\}$.
 - For instance, the classes may represent different levels of truthfulness.
 - We have a training dataset $\mathbb{D} \subset \mathbb{X} \times \mathbb{C}$, that consists of labeled text examples $\langle x, c \rangle$.

The Naive Bayes method for text classification

- One way to identify fake news is to rephrase the problem in terms of **text classification**:
 - We are given some text $x \in \mathbb{X}$, where $\mathbb{X}$ is our *input space*, and a fixed set of classes $\mathbb{C} = \{c_1, c_2, \dots, c_j\}$.
 - For instance, the classes may represent different levels of truthfulness.
 - We have a training dataset $\mathbb{D} \subset \mathbb{X} \times \mathbb{C}$, that consists of labeled text examples $\langle x, c \rangle$.
 - Our task is to learn, from the training data, a **classification function**:

$$\gamma : \mathbb{X} \rightarrow \mathbb{C}$$

The Naive Bayes method for text classification

- In the **Naive Bayes** method, we learn the probability of a document x being in class C :

$$\underbrace{\hat{P}(c|x)}_{\text{Posterior}} \propto \underbrace{\hat{P}(c)}_{\text{Prior}} \underbrace{\prod_{k=1}^{n_x} \hat{P}(t_k|c)}_{\text{Likelihood}}$$

n_x : number of x
 t_k : k -th term in x

The Naive Bayes method for text classification

- In the **Naive Bayes** method, we learn the probability of a document x being in class $\mathcal{C}$:

$$\underbrace{\hat{P}(c|x)}_{\text{Posterior}} \propto \underbrace{\hat{P}(c)}_{\text{Prior}} \underbrace{\prod_{k=1}^{n_x} \hat{P}(t_k|c)}_{\text{Likelihood}}$$

n_x : number of x
 t_k : k -th term in x

- Since we are interested in predicting the most likely class, we use the *maximum a posteriori* (MAP) estimator:

$$c_{map} = \arg \max_{c \in \mathcal{C}} \left(\log \hat{P}(c|x) \right)$$

The Naive Bayes method for text classification

- In the **Naive Bayes** method, we learn the probability of a document x being in class $\mathcal{C}$:

$$\underbrace{\hat{P}(c|x)}_{\text{Posterior}} \propto \underbrace{\hat{P}(c)}_{\text{Prior}} \underbrace{\prod_{k=1}^{n_x} \hat{P}(t_k|c)}_{\text{Likelihood}}$$

n_x : number of x
 t_k : k -th term in x

- Since we are interested in predicting the most likely class, we use the *maximum a posteriori* (MAP) estimator:

$$c_{map} = \arg \max_{c \in \mathcal{C}} \left(\log \hat{P}(c|x) \right)$$

Numerically stable

The Naive Bayes method for text classification

- Here, the prior is simply the relative frequency of the classes in the training data.
- The likelihood is computed as the relative frequency of the term within the class:

$$\hat{P}(t | c) = \frac{T_{ct} + 1}{\sum_{t' \in V} T_{ct'} + 1}$$

T_{ct} : occurrences of term t in class c
 V : training vocabulary

- We implement Laplace smoothing to avoid issues during inference, caused by terms not present in the training data.

Implementation of Naive Bayes in R

- Organizing and preparing the data:
 - Structure of preference → **dataframes**

Label	Text
1	Says the Annies List political group supports...
2	When did the decline of coal start? It started...
3	Hillary Clinton agrees with John McCain...
⋮	⋮

Implementation of Naive Bayes in R

- **Organizing and preparing the data:**
 - Structure of preference → **dataframes**
 - Stratified train-validation split → generate training and validation datasets

```
# Draw a random, stratified sample including p percent of the data  
idx.train <- createDataPartition(y = fakeNewsTrain$Labels, p = 0.8, list = FALSE)
```

```
# Training set with p = 0.8  
trainData <- fakeNewsTrain[idx.train, ]
```

```
# Validation set with p = 0.2  
validationData <- fakeNewsTrain[-idx.train, ]
```

Implementation of Naive Bayes in R

- **Organizing and preparing the data:**
 - Structure of preference → **dataframes**
 - Stratified train-validation split → generate training and validation datasets
 - Data processing → cleaning and preparing the text sequences

```
process_data <- function(data_var) {  
  result <- data_var$Text # Init variable  
  result <- gsub("[[:punct:]]+", " ", result) # Remove punctuation  
  result <- tolower(result) # Turn everything into lowercase  
  result <- strsplit(result, split=" ") # Split by whitespaces  
  # Additional techniques: remove stopwords, stemming, lemmatization  
  # ...  
  result  
}
```

Implementation of Naive Bayes in R

- **Building the Naive Bayes model :**
 - Initializing of model vocabulary → **dataframe** with known terms

```
generate_numerical_mapping <- function(termsVar){  
  flatTerms <- unlist(termsVar) # Single vector with all the terms  
  uniqueTerms <- unique(flatTerms) # Unique words  
  uniqueNum <- length(uniqueTerms) # Number of unique terms  
  d <- data.frame(Term = uniqueTerms, Id = c(1:uniqueNum)) # Build mapping  
  d  
}  
numMapping <- generate_numerical_mapping(trainProcessed)
```


Implementation of Naive Bayes in R

- **Building the Naive Bayes model :**
 - Initializing of model vocabulary → **dataframe** with known terms
 - Defining the data structure for the prior → **dataframe** with class labels and relative frequency

Label	Relative frequency
0	0.32
1	0.25
⋮	⋮

Implementation of Naive Bayes in R

- **Building the Naive Bayes model :**
 - Initializing the model vocabulary → **dataframe** with known *terms*
 - Defining the data structure for the prior → **dataframe** with *class labels* and relative frequency
 - Defining the data structure for the likelihood → **matrix** with the likelihood of each known *term* for each possible *class*

$$\begin{array}{c} \text{Classes} \\ \left[\begin{array}{cccc} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{array} \right] \end{array}$$

Terms

Implementation of Naive Bayes in R

- **Training the model :**
 - Estimating the prior → count number of occurrences for each class and divide by the total number of samples

```
train_prior <- function (dataVar){  
  uniqueLabels <- unique(dataVar$Labels) # Get all the labels from the data  
  labelCounts <- table(dataVar$Labels) # Get counts  
  labelData <- as.data.frame(labelCounts) # Turn into dataframe  
  labelData$RelFreq <- labelData$Freq / nrow(dataVar) # Get relative frequency  
  labelData  
}  
  
# Estimate prior from our training data  
priorBayes <- train_prior(trainData)
```

Implementation of Naive Bayes in R

- **Training the model :**
 - Estimating the prior → count number of occurrences for each class and divide by the total number of samples
 - Estimating the log-likelihood → from occurrences of terms and Laplace smoothing

```
train_log_likelihood <- function(dataVar, mappingVar, processingFunc){  
  uniqueLabels <- unique(dataVar$Labels) # Get unique labels  
  termsM <- matrix(0,nrow=nrow(mappingVar),ncol=length(uniqueLabels)) # Init counts  
  for (i in uniqueLabels){  
    # Get data in the same class, process text, get counts for each term...  
    termsM[mappedText$Id,labelIdx] = mappedText$Freq # Update counts matrix  
  }  
  topTerm <- termsM + 1 # Laplace smoothing  
  M <- log(topTerm) - log(colSums(topTerm)) # Get log-likelihood matrix  
  M}
```

Implementation of Naive Bayes in R

- Using the model for inference :

```
naiveBayesPredict <- function(dataVar, priorVar, logLikeliVar, mappingVar,
                              processingFunc=process_data){
  processedText <- processingFunc(dataVar) # Process text input
  predClass <- 0*c(1:nrow(dataVar)) # Init results
  for (i in c(1:nrow(dataVar))){ # Operate separately for each input case
    inputText <- processedText[i] # Get input data
    countsTerms <- as.data.frame(table(inputText)) # Counts for each term and map
    names(countsTerms)[names(countsTerms) == "inputText"] <- "Term"
    mappedText <- merge(mappingVar, countsTerms, by = "Term")
    weightsTerms <- logLikeliVar[mappedText$Id,1:nrow(priorVar)] # Get weights
    logLikelyTerms <- colSums(mappedText$Freq*weightsTerms) # Get log-likelihood
    logPosteriorTerms <- logLikelyTerms + log(priorVar$RelFreq) # Add log-prior
    predClass[i] = which.max(logPosteriorTerms) - 1 # Get predicted class with MAP
  }
  predClass}
```

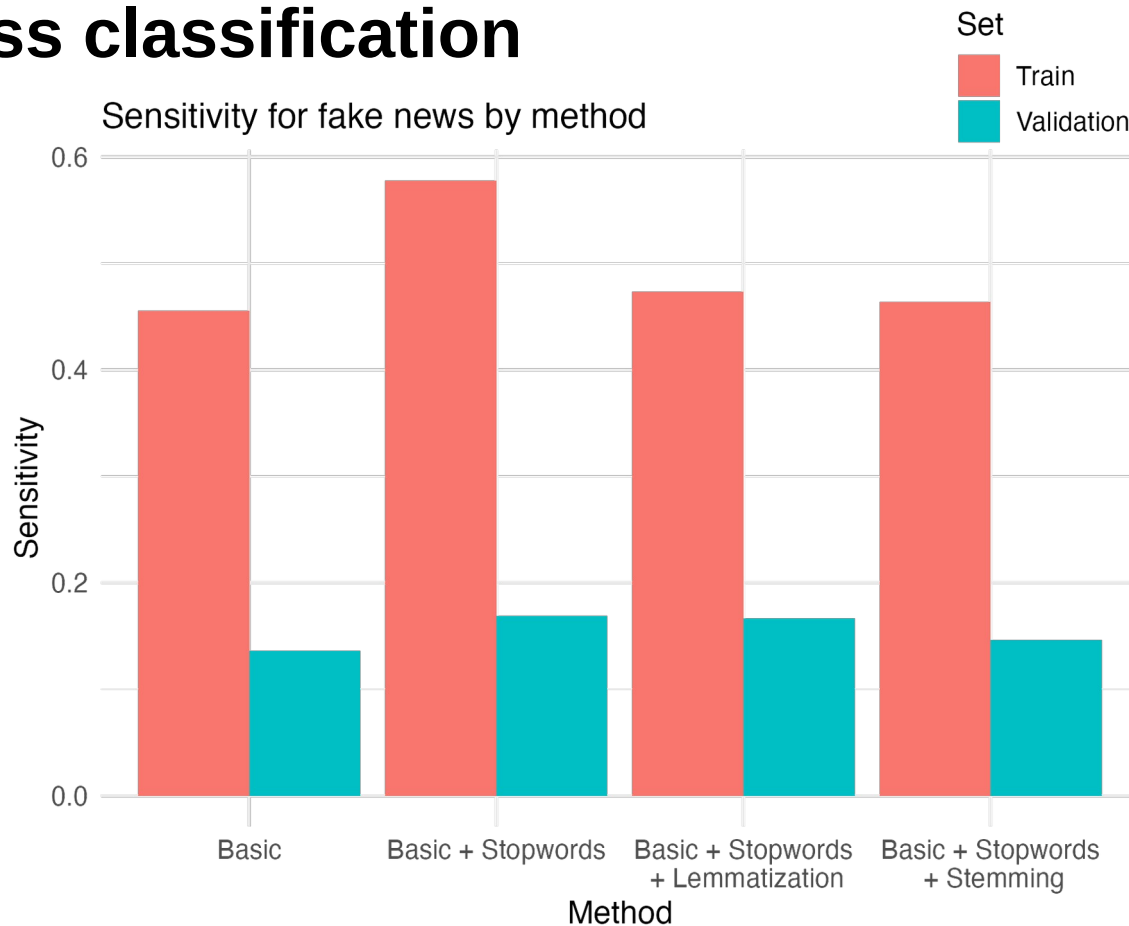
Results: multi-class classification

Sensitivity:

$$\frac{TP}{TP + FN}$$

Poor sensitivity in multi-class classification and even worse performance in the validation set.

In data preprocessing, stop words have a better effect, while other methods have little impact

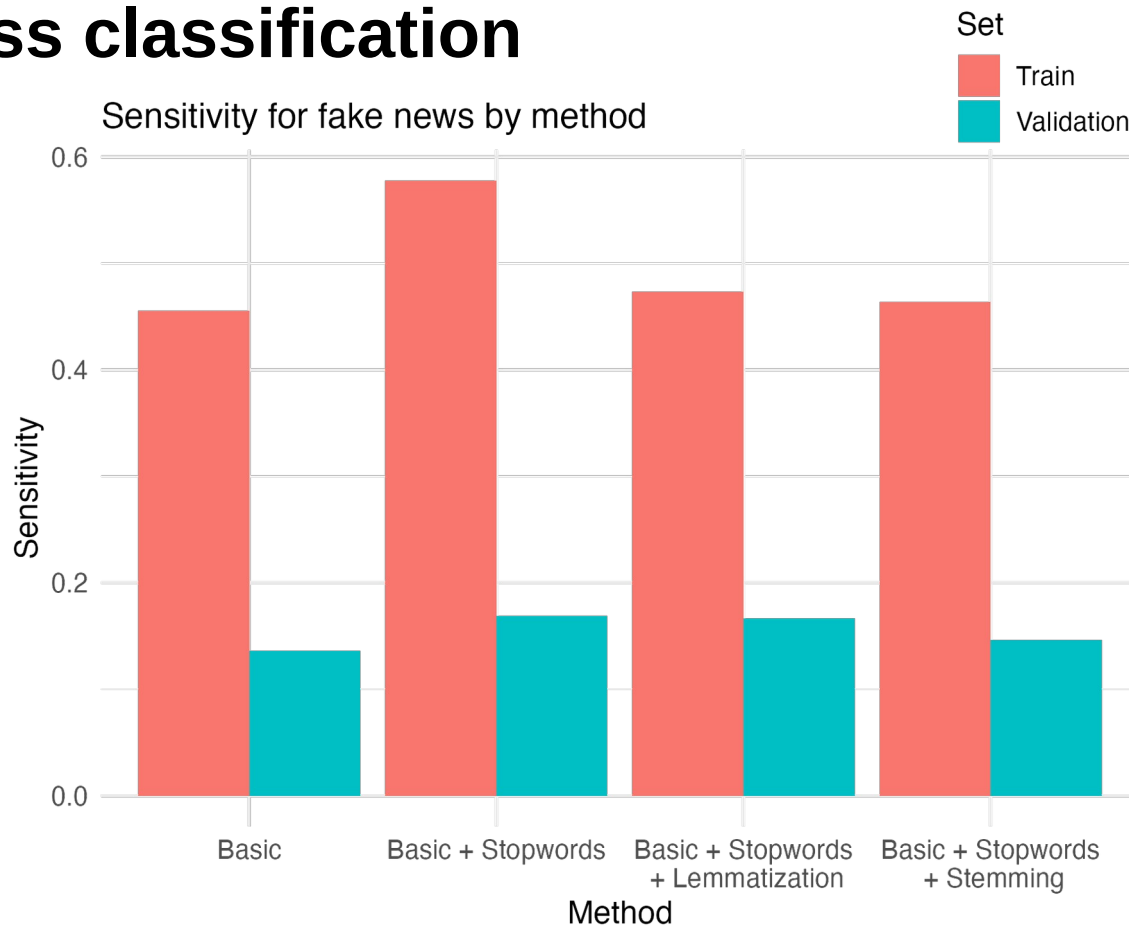


Results: multi-class classification

Sensitivity:

$$\frac{TP}{TP + FN}$$

Neglecting more
correct labels in
prediction

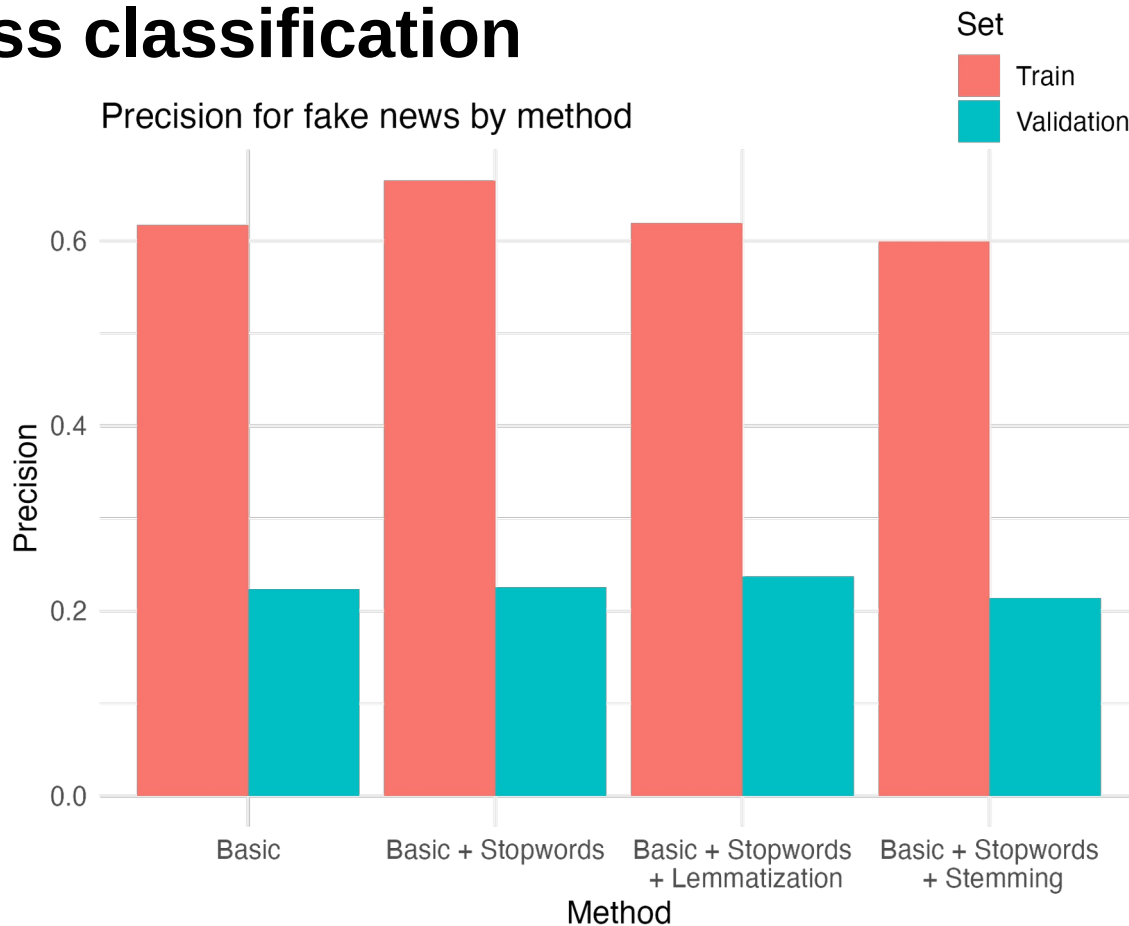


Results: multi-class classification

Precision:

$$\frac{TP}{TP + FP}$$

Precision performance is better than sensitivity, and the impact of data preprocessing is not significant

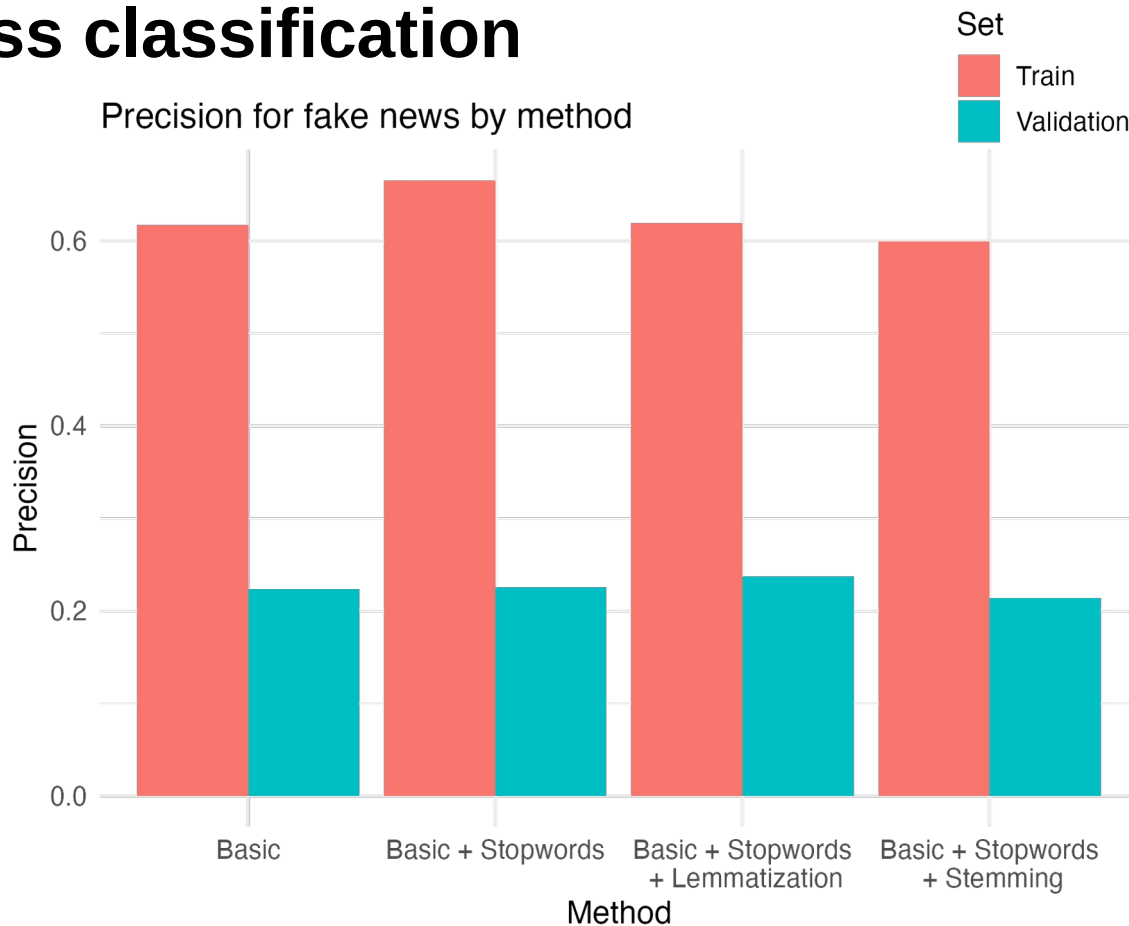


Results: multi-class classification

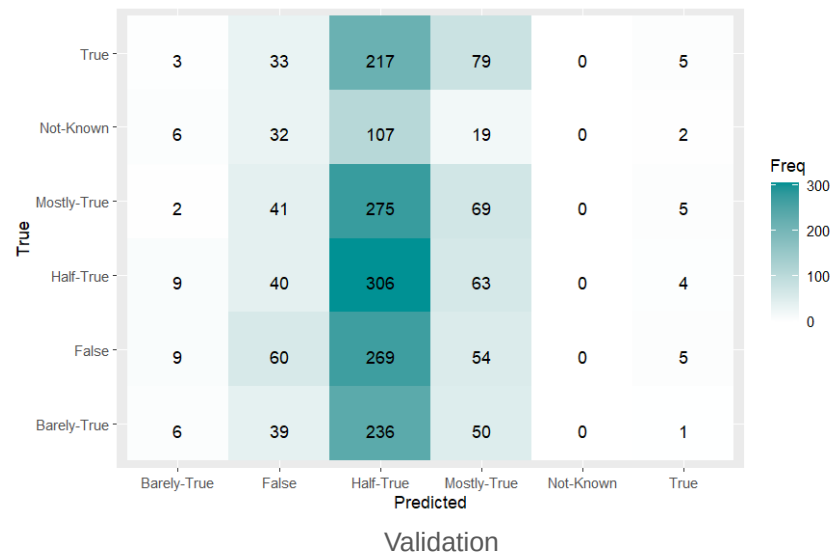
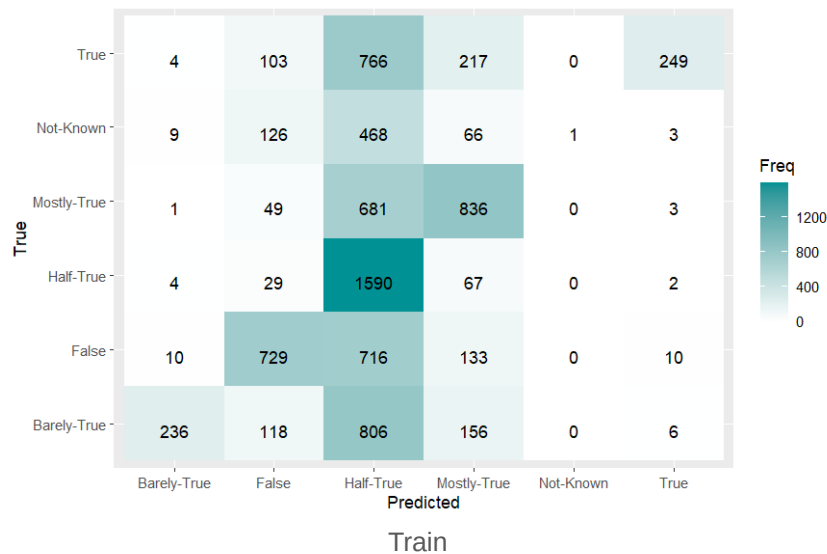
Precision:

$$\frac{TP}{TP + FP}$$

Compared to ignoring correct predictions, there are fewer incorrect predictions in predictions.



Possible cause analysis

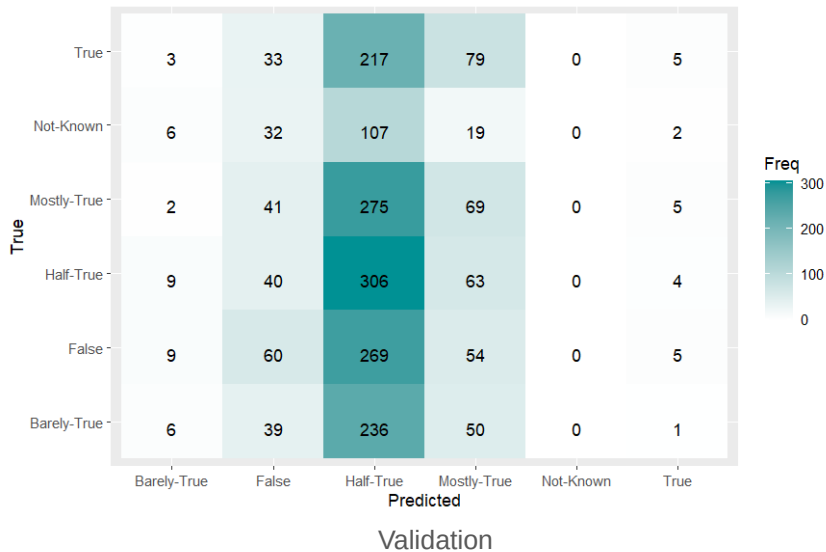


A large number of errors occur in predicted 'Half True' label.

The farther away the predicted labels are (with the greatest difference), predictions are more difficult, leading to mistakes. This can explain why precision performs much better than sensitivity.

For all labels, they are more likely to be predicted as middle labels, and the distribution of intermediate labels is not significantly higher than others. This is the reason for the low sensitivity and precision.

Possible cause analysis



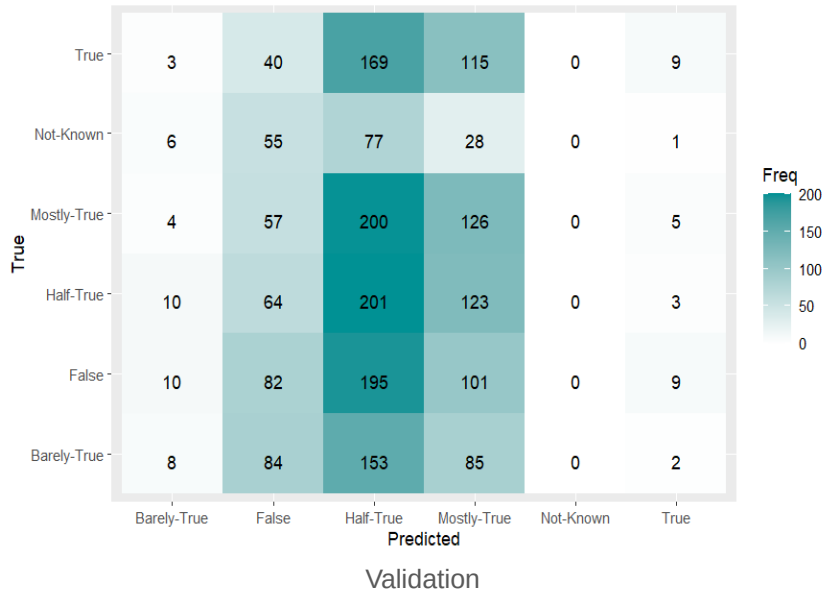
The reason for the poor performance of the test set is that it is too small

In fact, with fewer Half-True labels, their predictions may all be worse.

For the labels with the largest distribution, even small changes can have a huge impact, which may be due to the influence of prior.

Possible cause analysis

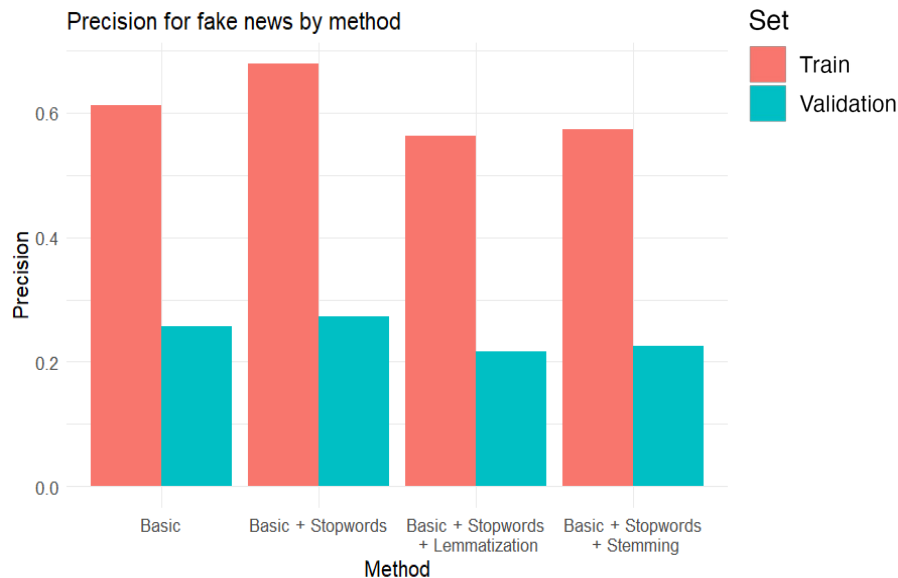
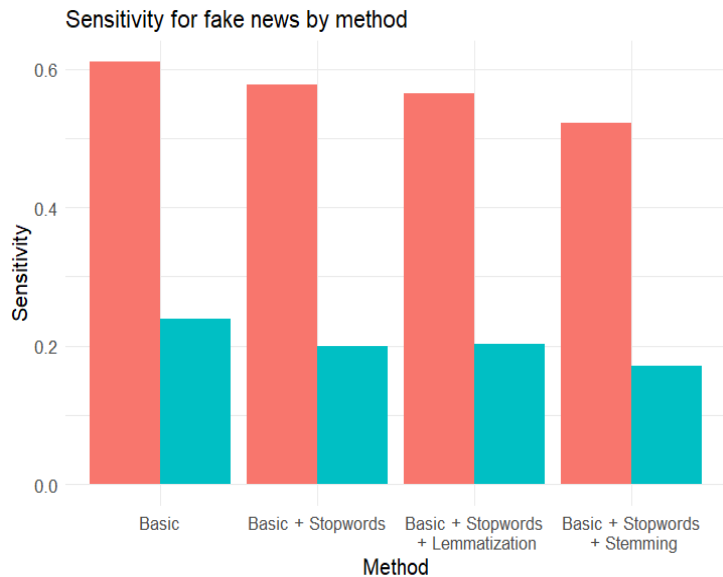
In this section, we deleted 5% of the samples with label 2 (half-true).



When the sample distribution is more uniform, the prediction performance is better.

Possible cause analysis

In this section, we deleted 5% of the samples with label 2 (half-true).



When the sample distribution is more uniform, the prediction performance is better.

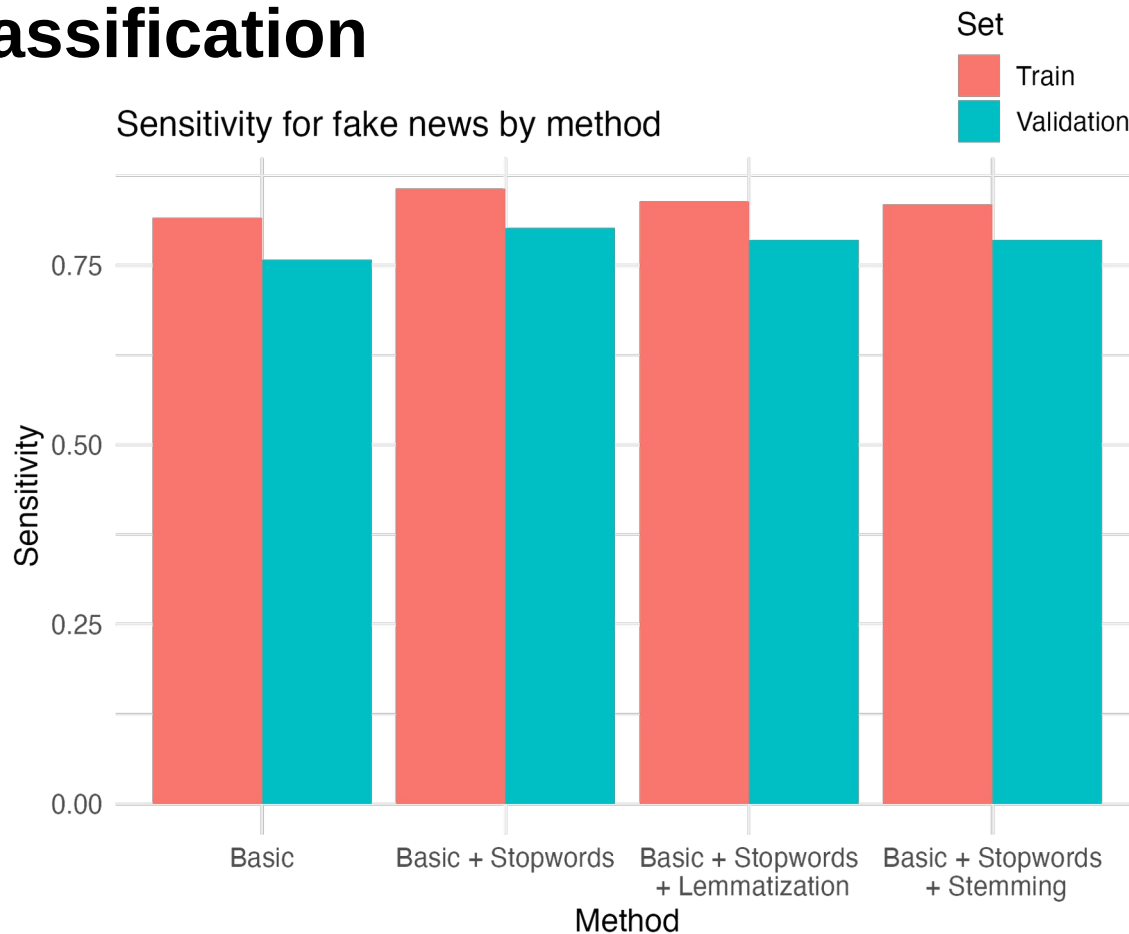
Results: binary classification

Sensitivity:

$$\frac{TP}{TP + FN}$$

In binary classification,
sensitivity performs
well.

The role of
preprocessing is still
limited

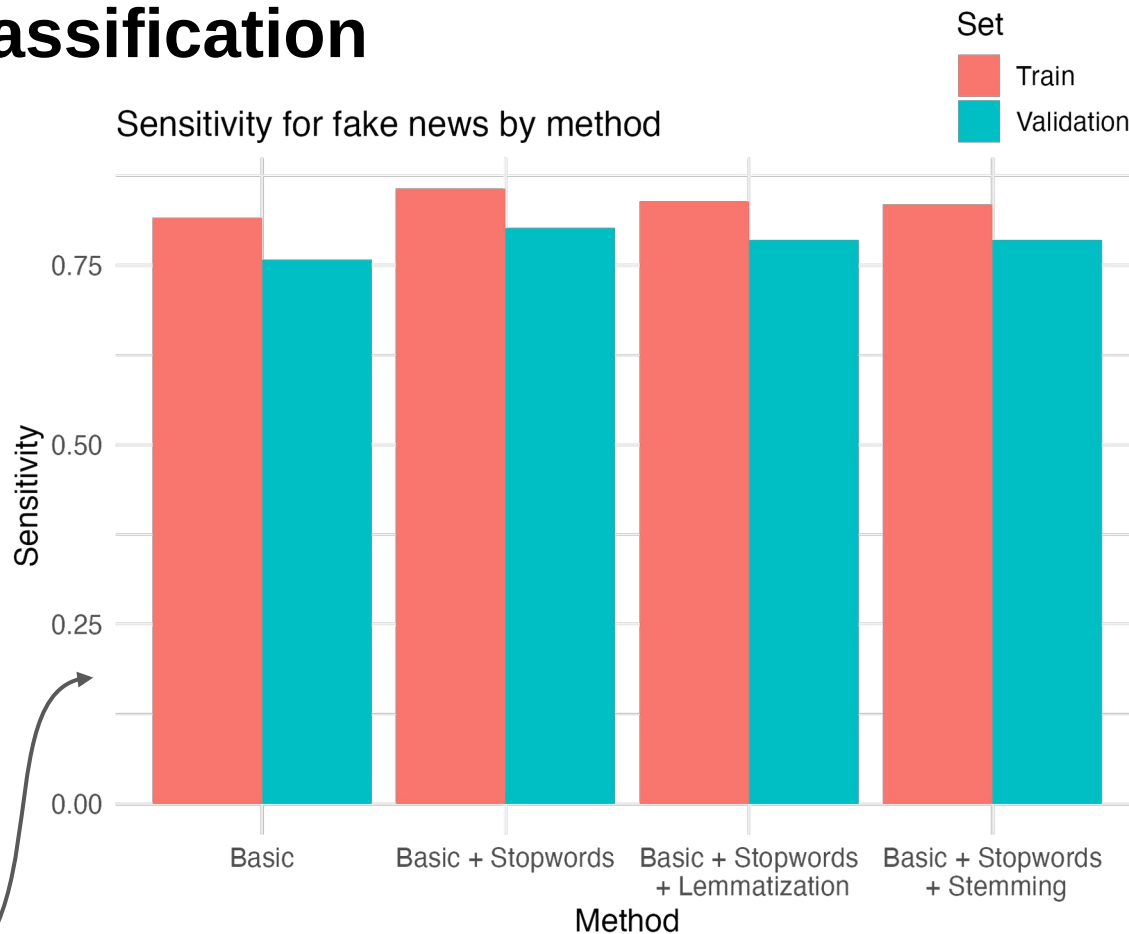


Results: binary classification

Sensitivity:

$$\frac{TP}{TP + FN}$$

We can detect more than 75% of the fake news!



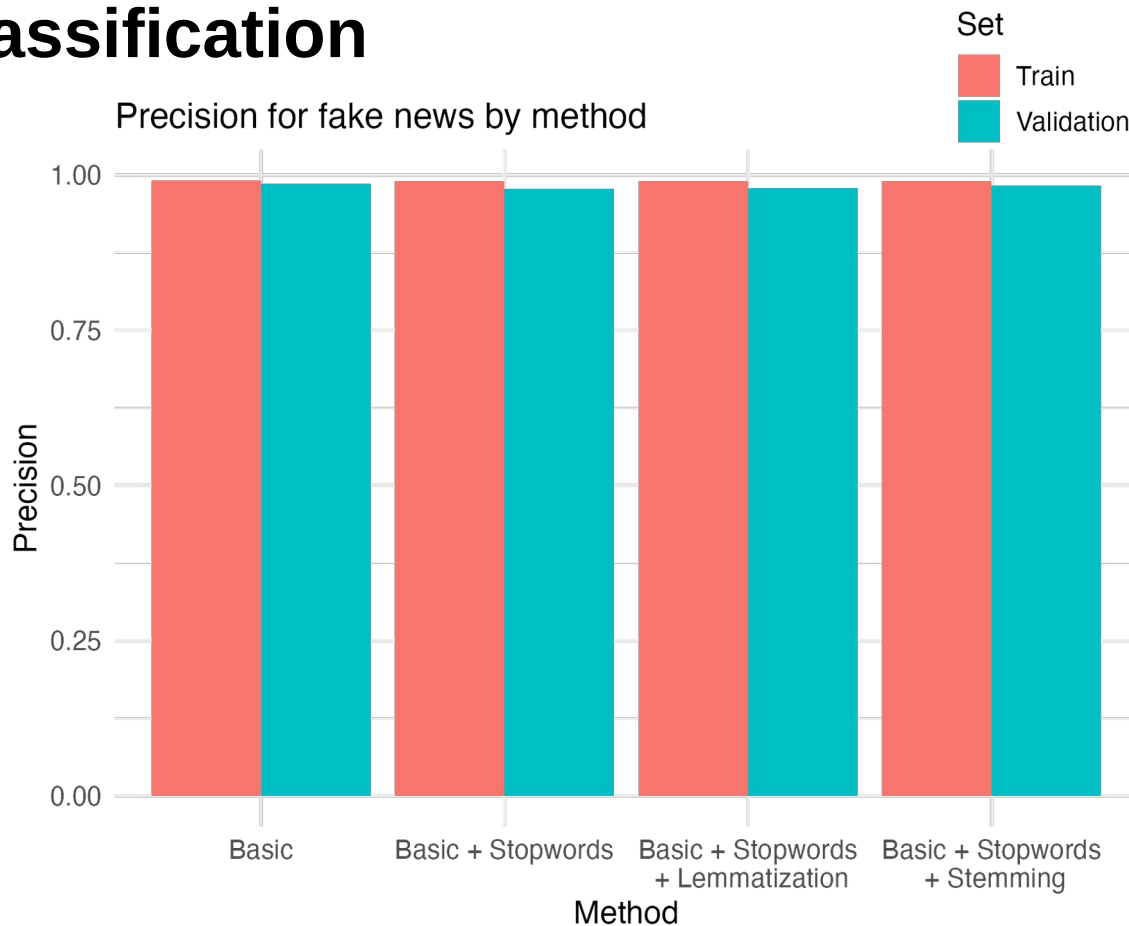
Results: binary classification

Precision:

$$\frac{TP}{TP + FP}$$

In binary classification,
the precision is almost 1.

The model almost never
makes incorrect
predictions

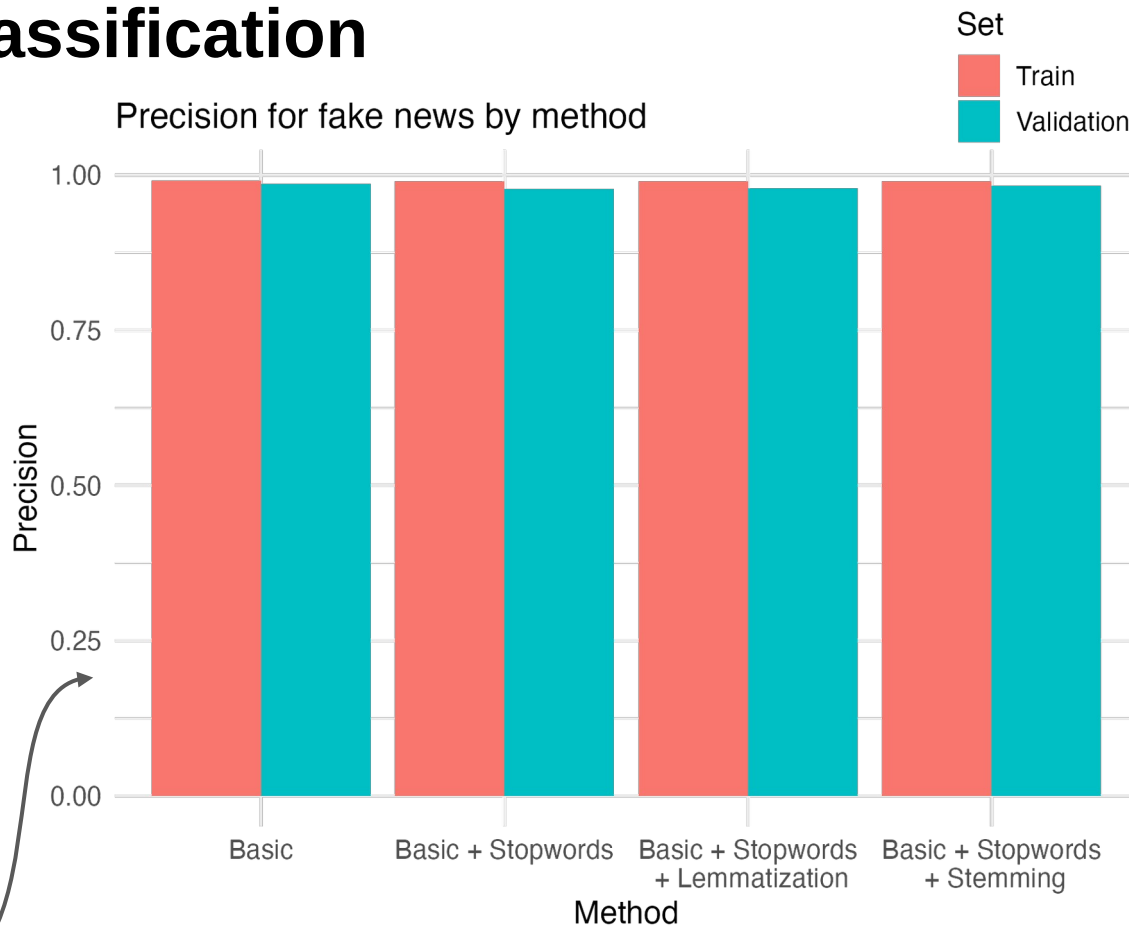


Results: binary classification

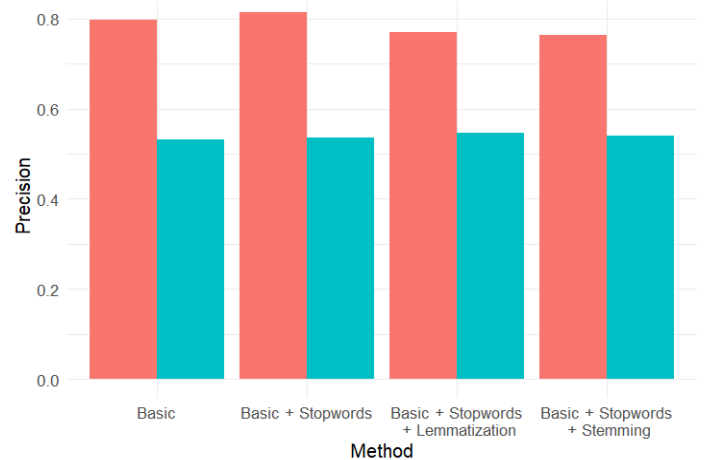
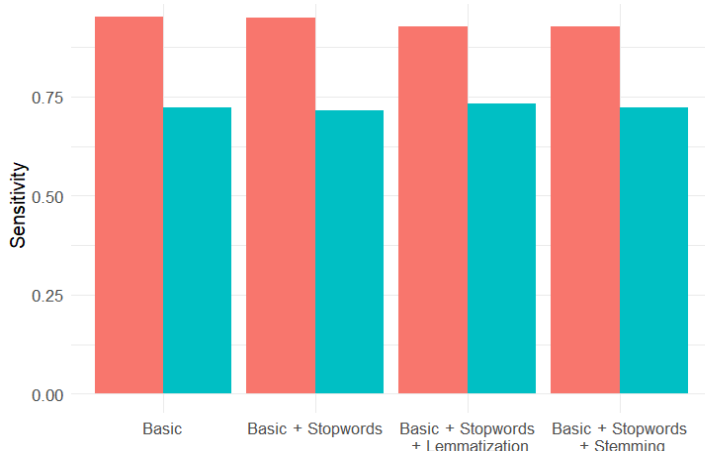
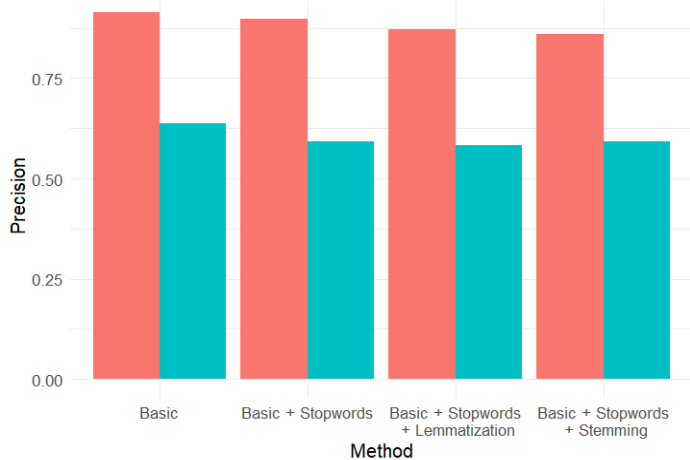
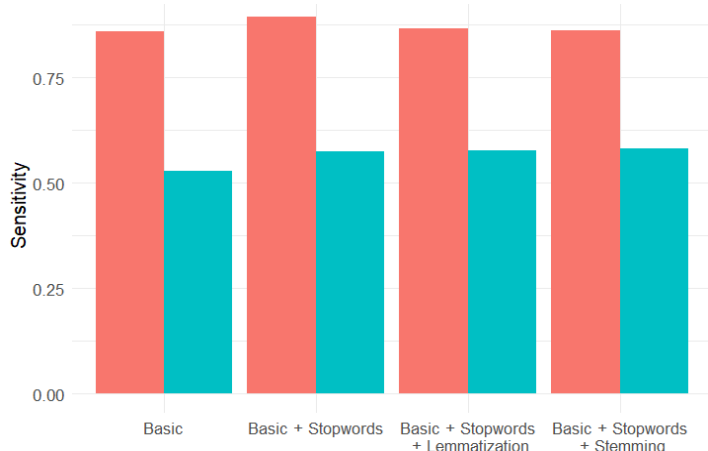
Precision:

$$\frac{TP}{TP + FP}$$

When we classify news as fake, we are almost always right!



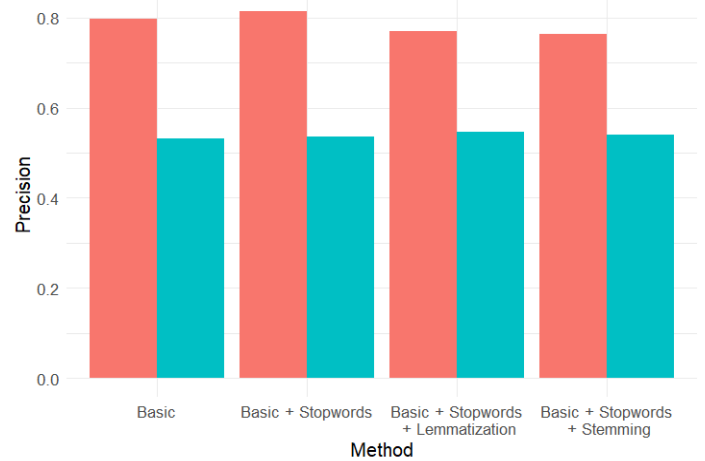
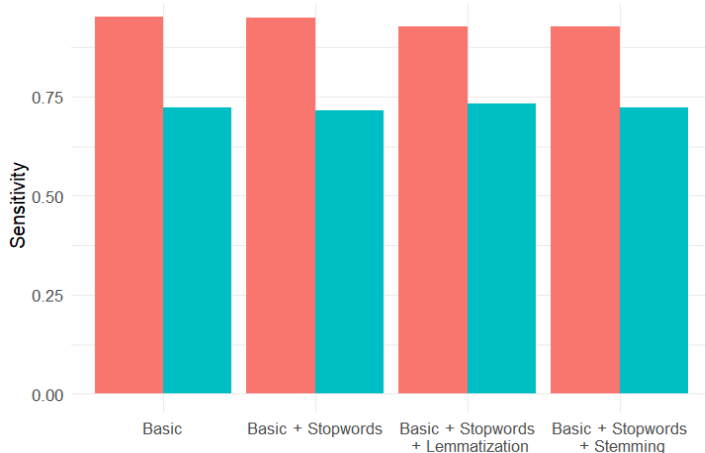
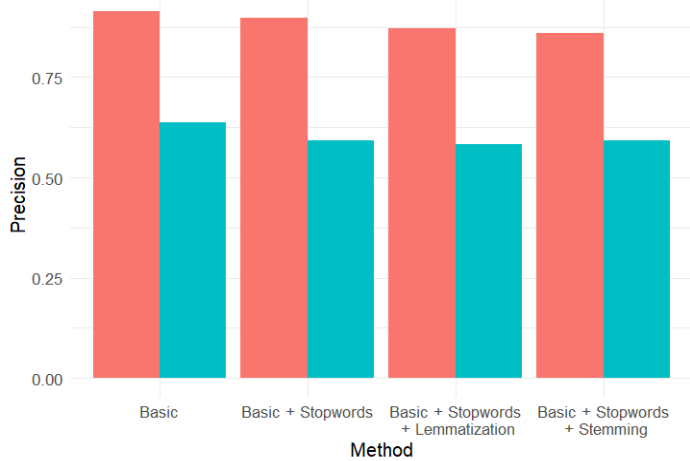
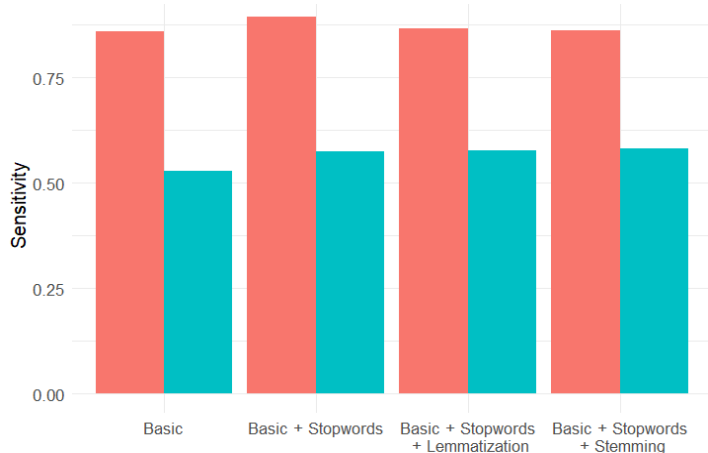
Results: reducing multi-class to binary classification



- The two images on the left show binary classification using labels **0 (barely-true)** and **5 (true)**
- The two images on the right show binary classification using labels **2 (half-true)** and **3 (mostly-true)**



Results: reducing multi-class to binary classification



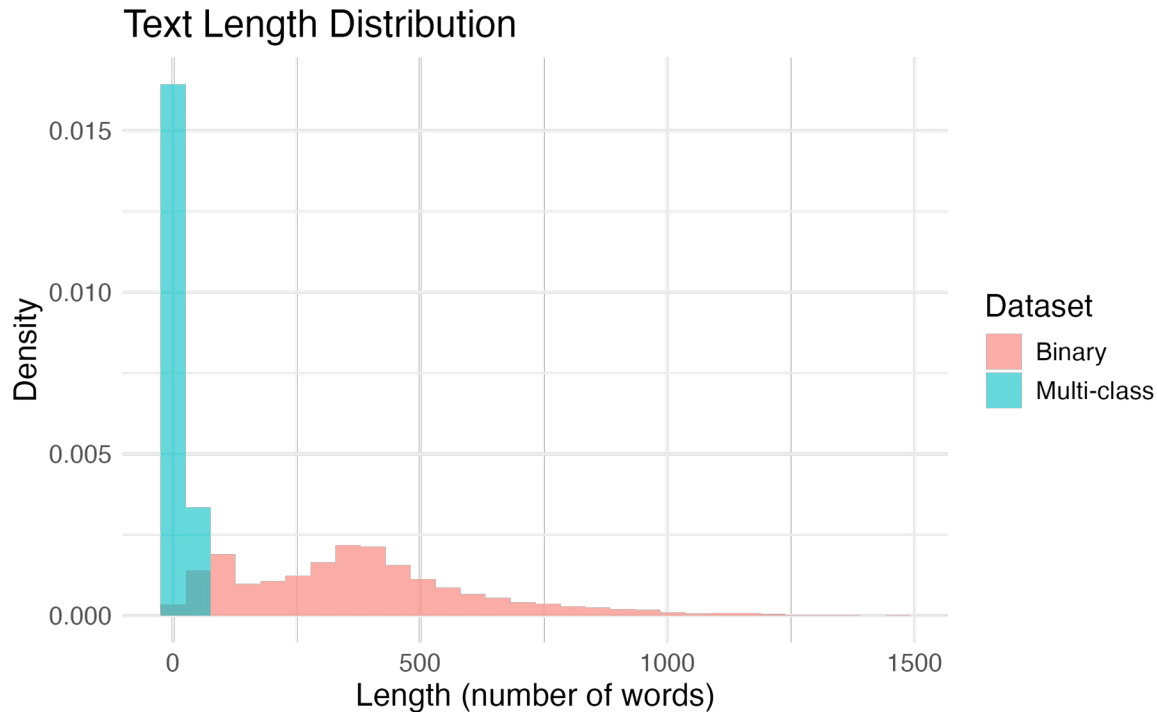
Due to significant prior knowledge, it performs well. The performance of training sets with more samples is similar to that in binary classification sets

Set

Train

Validation

Discussion: text length distribution



- The length of the inputs is significantly lower for the multi-class dataset, compared to the binary case.
- This may cause the predictions to rely more heavily on the prior for the multi-class data.

Concluding remarks

- Tackled the problem of **fake news** classification using the **Naive Bayes** method.
- Applied the method to **two different datasets**:
 - Fake News Content Detection (**multi-class** data)
 - Fake News Classification (**binary** data)
- Tested several **text processing** techniques in order to improve the performance of the model:
 - Stopword removal
 - Stemming
 - Lemmatization
- The method displayed significant limitations in the multi-class scenario, but performed very well on binary data.

Thank you

for your attention!

Any Questions?

Group members:

Hooman Sabzi, Franco Aquistapace, Daize Li

